Spring Framework

Spring MVC

Rappel des objectifs du cours

- Créer une application Web MVC simple
- Créer des templates simples avec Thymeleaf
- Manipuler nos données via une page HTML

Spring Framework

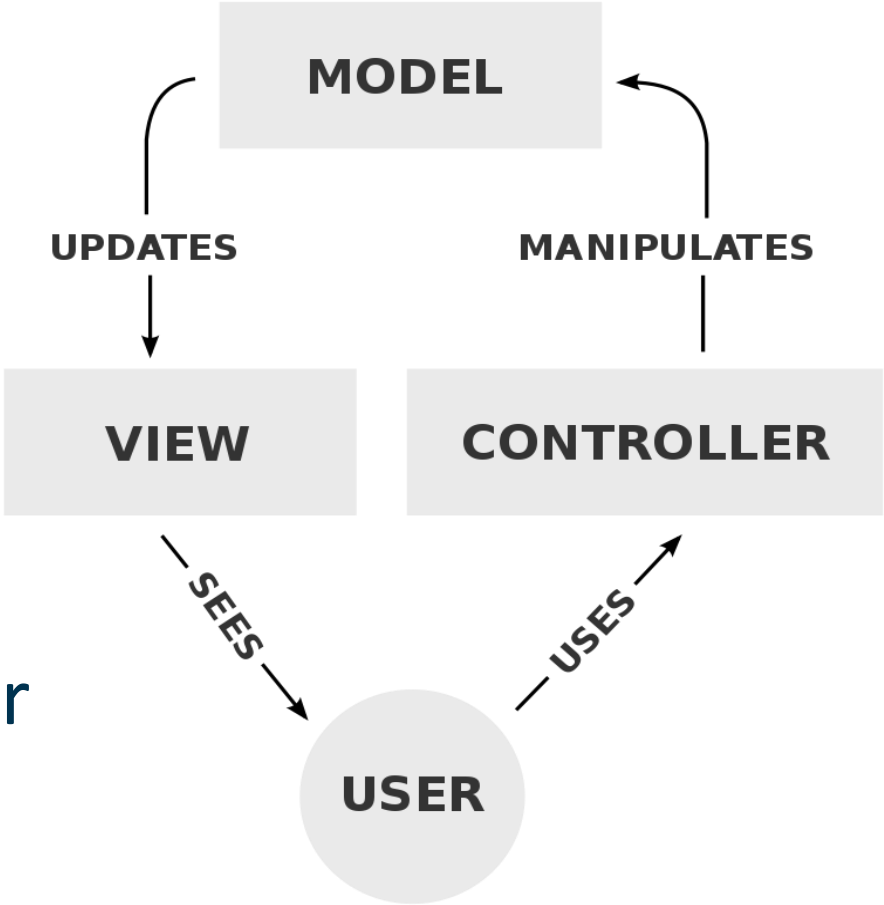
Partie 1 – Intro à l’architecture MVC et Spring MVC

Architecture MVC

- MVC (Model-View-Controller) est un **pattern d'architecture** principalement utilisé pour développer les applications web.

- **Son but est de diviser notre application en sous-parties**

- Réduire l'interdépendance entre vue et modèle grâce au contrôleur



Architecture MVC

Model

- **Représente les données** et rien d'autre.
- Ici on ne parle pas (que) de nos entités JPA (concept différent).

View

- **La représentation** des données qui sera montrée à l'utilisateur.
- Ce qui est visible à l'écran (JSP, HTML enrichi, …)

Controller

- **Le traitement** des actions de l'utilisateur.
- Intermédiaire entre la vue et le modèle.

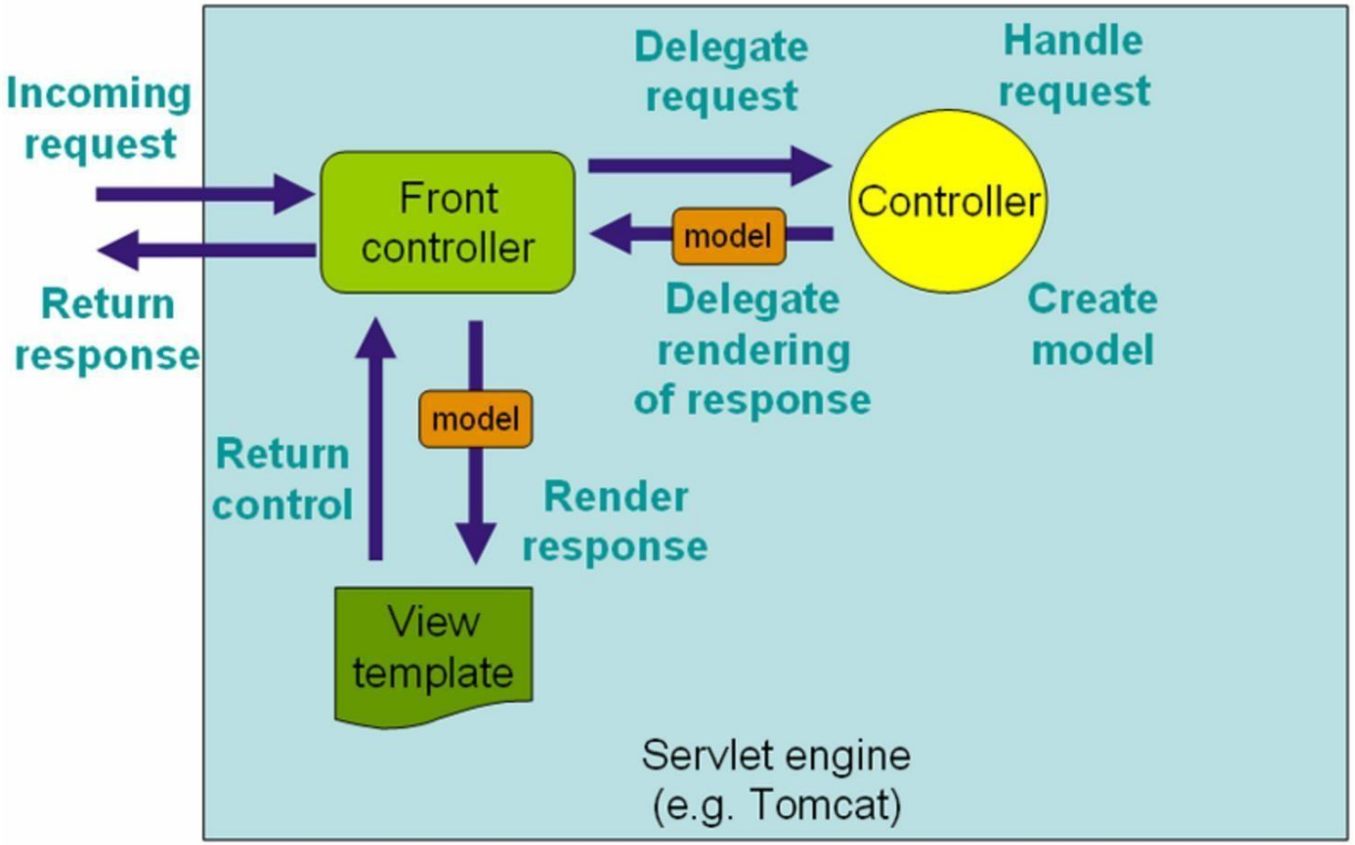
Spring MVC

Spring MVC

- Spring va nous faciliter la mise en place du pattern MVC.
- Il rajoute le concept de Front Controller qui va se charger d’aiguiller la requête de l’utilisateur vers le bon Controller.

Architecture MVC

MVC dans Spring MVC :



Spring Framework

Partie 2 – Développer une application web MVC avec Spring

Spring MVC – Pratique

Construisons ensemble une application web avec Spring MVC.

Objectifs :

- Mettre en place chaque élément nécessaire à une application Spring MVC.
- Reprendre la partie Data du cours Spring Data.

Spring MVC

Spring MVC – Pratique – Initialisation du projet

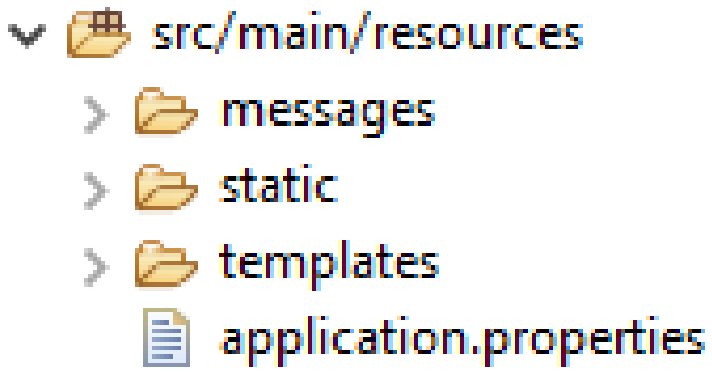
Étape 1 : Créer le projet et copier la partie persistence

On va partir sur un nouveau projet.

- Nommer le projet par exemple *SpeciesMVC*
- **Starters** :
  - *Web,*
  - *Thymeleaf,*
  - *Devtools,*
  - *Data Jpa,*
  - *Driver BDD (MySQL, MariaDB ou Postgres)*
- Copier les packages des classes qu'on a créé lors des TP 3, 4, 5 ainsi que la datasource de application.properties

Spring MVC – Pratique – Initialisation du projet

Dans le dossier **resources**, on a des dossiers en plus :



- **static** : contient les ressources statiques du site (css, images, js, etc)
- **templates** : contient les Vues (donc des fichiers HTML)
- **messages** : contiendra nos fichiers de traduction

Spring MVC – Pratique – Mise en place des Controllers

Le premier élément à mettre en place sera un controller.

- C’est une classe java annotée **@Controller**
- Chaque méthode de cette classe aura un **mapping** qui va **<u>intercepter une URL précise</u>.**
- Ces méthodes renvoient **le nom de la vue à afficher.**

Spring MVC – Pratique – Mise en place des Controllers

Exemple d'un controller qui possède une méthode qui va afficher la liste des *Species* :

```
@Controller
public class SpeciesController {

    @Autowired
    private SpeciesRepository speciesRepository;

    @GetMapping("/specie")
    public String listSpecies(Model model) {
        List<Species> species = speciesRepository.findAll();
        model.addAttribute("species", species);
        return "list_specie";
    }
}
```

Spring MVC – Pratique – Mise en place des Controllers

@Controller

@GetMapping
Intercepte les GET sur /species

```
@Controller
public class SpeciesController {

    @Autowired
    private SpeciesRepository speciesRepository;

    @GetMapping("/species")
    public String listSpecies(Model model) {
        List<Species> species = speciesRepository.findAll();
        model.addAttribute("species", species);
        return "list_specie";
    }
}
```

Met à jour le "Model"

Retourne le nom de la vue

Spring MVC – Pratique – Mise en place des Controllers

```
@Controller
public class SpeciesController {
    @Autowired
    private SpeciesRepository speciesRepository;
    // { … }
```

Remarque sur l'utilisation du Repository

- Ici on utilise directement le *Repository* dans notre controller.
- C'est une bonne pratique d'appeler plutôt une classe de type **Service** qui va se charger d'utiliser nos repository et d'y ajouter de la logique "métier" si besoin.
- Pour garder notre projet simple, on va faire sans cette couche Service.

Spring MVC – Pratique – Mise en place des Controllers

```
public String listSpecies(Model model) {
    // { … }
    model.addAttribute("species", species);
    // { … }
```

Précisions sur le “Model” :

- Ce sont nos **données nécessaires** pour afficher notre **vue**
- **Il est toujours possible de passer le Model en paramètre des fonctions d’un Controller**
- Le controlleur rajoute ici l’attribut “species” au modèle
- Cet attribut **sera lu par Thymeleaf** pour afficher son contenu.

Spring MVC – Pratique – Mise en place des Controllers

Avant d'aller plus loin, quelques remarques sur les URL :

- Une requête HTTP **GET** est destiné à afficher une page
- Une requête HTTP **POST** indique un traitement à effectuer à partir des données de la Vue (en général un formulaire)
- Pas de PUT en MVC.

Spring MVC – Pratique – Mise en place des Controllers

Ce qui suit sont des bonnes pratiques plutôt que des obligations :

- **GET /item/** : indique une liste à afficher (ex : /specie/)

- **GET /item/action/id** : indique une action à faire un id donnée
  - GET /specie/2 : affiche la specie d'id 2 (action est omise)
  - GET /specie/delete/2 : détruit la specie d'id 2

- **POST /item/action/id** : provient d'un formulaire
  - POST /specie/ : crée une Specie (depuis un formulaire)
  - POST /specie/2 : met à jour la Specie d'id 2 (depuis un formulaire)

Spring MVC – Pratique – Mise en place des Controllers

Méthode “initUpdate” sur **GET /species/{id}**

Va intercepter
GET /species/{id}

{id} est une
PathVariable

```
@GetMapping("/species/{id}")
public String initUpdate(@PathVariable("id") Integer id, Model model) {
    Optional<Species> species = speciesRepository.findById(id);
    if (species.isPresent()) {
        model.addAttribute(species.get());
        return "update_species";
    }
    return "error";
}
```

Retourne la vue
« **update_species** »
si entité trouvée

Spring MVC – Pratique – Mise en place des Controllers

Méthode “initCreate” sur **GET /species/create**

Va intercepter
GET /species/create

```
@GetMapping("/species/create")
public String initCreate(Model model) {
    model.addAttribute(new Species());
    return "create_species";
}
```

Objet « vide »

Retourne la vue
« **create_species** »

Spring MVC – Pratique – Mise en place des Controllers

Étape 2 : Créer les Web Controllers

- Pour chaque entité, créer dans un package controller le **controller** correspondant.

- Ajouter les méthodes suivantes
  - **GET** sur **/nom-entité** (ex. /species) ➔ retourne la liste des entités
  - **GET** sur **/nom-entité/{id}** (ex. /species/2) ➔ retourne l'entité d'id "id"
  - **GET** sur **/nom-entité/create** (ex. /species/create) ➔ retourne la vue "*nom-entité*_create"

- Vous pouvez ajouter des fichiers html simples correspondant à ces vues dans **resources/template** afin de tester vos controleurs.

Spring MVC – Pratique – Les Vues

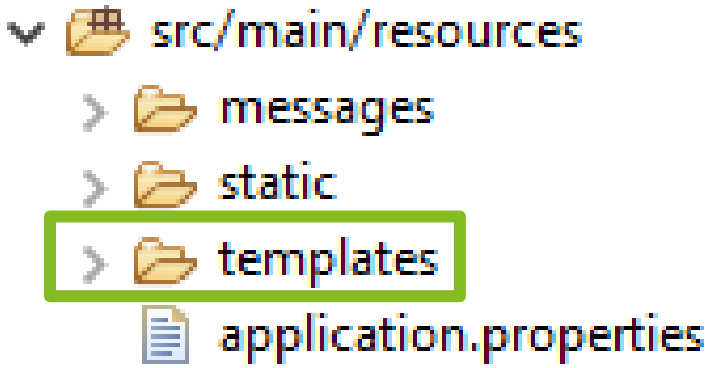
La prochaine étape va être d'écrire nos vues.

- Dans notre cas, **Thymeleaf** s'en charge
- C'est un **générateur de view template**
- Il va nous permettre **d'enrichir notre HTML** via des attributs et balises spéciaux.

Spring MVC – Pratique – Les Vues

Rappel :

- Nos vues sont placées dans le dossier **templates**



Spring MVC – Pratique – Les Vues

```
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <link rel="stylesheet" th:href="@{/resources/css/main.css}"/>
  <meta charset="utf-8">
  <title>Liste des Espèces</title>
</head>
<body>
<table class="mainTable">
  <thead>
  <!-- … -->
  </thead>
  <tbody>
  <tr th:each="speciesItem : ${speciesList}">
    <td th:text="${speciesItem.commonName}"/>
    <td th:text="${speciesItem.latinName}"/>
  </tr>
  </tbody>
</table>
</body>
```

Déclaration Thymeleaf

CSS

Boucle sur la liste de Species

Spring MVC – Pratique – Les Vues

```
<form th:object="${species}" th:action="@{/specie}" method="post">
  <input type="hidden" th:field="*{id}"/>
  <div>
    <label for="commonName">Nom Commun : </label>
    <input type="text" th:field="*{commonName}" id="commonName"/>
  </div>
  <div>
    <label for="latinName">Nom latin :</label>
    <input type="text" th:field="*{latinName}" id="latinName"/>
  </div>

  <div>
    <input type="submit" class="button" value="Sauver"/>
    <input type="button" class="button" value="Retour"
onclick="window.location='/species'"/>
  </div>
</form>
```

On utilise l'attribut « species » du modèle en tant qu'Objet du formulaire

*{*nomChamp*}
Le champ de l'objet sélectionné

Spring MVC – Pratique – Les Vues

Comment faire pour afficher une liste d'entités dans une liste déroulante ?

- L'ajouter dans le modèle

```
model.addAttribute(
    "speciesList",
    speciesRepository.findAll(Sort.by(Sort.Direction.ASC, "commonName"))
);
```

- Itérer dans le template

```
<div>
  <label for="species">Espèce :</label>
  <select th:field="*{species}" id="species">
    <option th:each="sp:${speciesList}" th:value="${sp.id}" th:text="${sp.commonName}"></option>
  </select>
</div>
```

Spring MVC – Pratique – Les Vues

- Doc Thymeleaf
- Doc Thymeleaf Spring

Spring MVC – Pratique – Les Vues

Étape 3 : Écrire nos Vues pour nos méthodes GET

- Pour chaque entité, **écrire la vue**
  - **GET /nom-entité :** affiche la liste des entités
  - **GET /nom-entité/{id}** : affiche un formulaire d'édition sur une entité
  - **GET /nom-entité/create** : affiche un formulaire de création sur l'entité (peut être copié/collé de update)

- Sur la liste d'entités, ajouter un lien vers l'update (exemple)

```
<td><a th:href="@{'/species/' + ${speciesItem.id}}">📝</a></td>
```

- Un bouton qui amène vers la page de création (exemple)

```
<button type="button" onclick="window.location='/species/create'">➕ Ajouter une Espèce</button>
```

Spring MVC – Pratique – Traitement des POST

Ensuite, on va écrire les méthodes du controller qui traitent les demandes POST (create, update).

```
@PostMapping("/species")
public String createOrUpdate(Species speciesItem) {
    this.speciesRepository.save(speciesItem);
    return "redirect:/species";
}
```

- Ici, la méthode « save » du repo fait la création et l'update.
- On redirige une fois le traitement terminé.

Spring MVC – Pratique – Traitement des POST

Méthode delete du contrôleur :

@PathVariable

```
@GetMapping("/species/delete/{id}")
public String delete(@PathVariable("id") Integer speciesId) {
    Optional<Species> speciesToDelete = this.speciesRepository.findById(speciesId);
    speciesToDelete.ifPresent(species -> this.speciesRepository.delete(species));
    return "redirect:/species";
}
```

- Ici on va chercher l'entité par son id et on vérifie si elle existe avant de la supprimer
- (On aurait pu écrire une méthode deleteById sur le repo)

Spring MVC – Pratique

Étape 4 : Ajouter les méthodes POST & le delete

- Ajouter les méthodes suivantes aux controllers
  - **POST** sur **/nom-entité** : action du formulaire de création/update
  - **GET** sur **/nom-entité/delete/{id}** : demande de suppression (pas de formulaire lié)

- Ajouter un bouton suppression sur la liste d'entités avec une boîte de dialogue de confirmation (exemple)

```
<a th:href="@{'/species/delete/' + ${speciesItem.id}}"
   onclick="return confirm('Êtes-vous sûr ?')">🗑</a>
```

Spring MVC – Pratique – Validation

Problématique :

- On peut créer des Personnes sans nom ou prénom, des Species sans nom…

Solution :

- ➡ Il faut effectuer une validation
- ➡ Faire une simple validation du formulaire sur l'interface n'est pas suffisant
- ➡ Il faut un mécanisme de validation côté java (serveur)

Spring MVC – Pratique – Validation

Validation côté java :

- Dépendance maven « spring-boot-starter-validation »

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

- Cela va aller chercher **Hibernate Validator Engine**

Spring MVC – Pratique – Validation

Pour les cas simples, on peut ajouter des annotations sur les champs de l'entité que l'ont veut vérifier :

```
@Column(length = 50)
@NotEmpty
@Size(max = 50)
private String lastname;

@Max(120)
@Min(0)
private Integer age;
```

Exemples :

- **@NotEmpty** : ne doit pas être null ou de longueur nulle
- **@Size(min=0, max=10)** : la longueur doit être comprise entre 0 et 10
- **@Min** et **@Max** : s'utilisent sur des nombres (sauf float)

Spring MVC – Pratique – Validation

Une fois cela fait, il faut ensuite déclencher la validation avec **@Valid** aux endroits où on veut vérifier :

```
@PostMapping("/person")
public String createOrUpdate(@Valid Person person, BindingResult result) {
    if (result.hasErrors()) {
        return "create_person";
    }
    this.personRepository.save(person);
    return "redirect:/persons";
}
```

@Valid

On retourne au formulaire si erreur

BindingResult ajouté (contient le résultat de la validation)

Spring MVC – Pratique – Validation

Du côté du template Thymeleaf :

```
<label for="firstname">Prénom : </label>
<input type="text" th:field="*{firstname}" id="firstname"/>
<span th:if="${#fields.hasErrors('firstname')}" th:errors="*{firstname}">Error</span>
```

th:if
Si le champ a une erreur, on affiche ce span

th:errors
Affiche toutes les erreurs sur le champ

→ Doc Thymeleaf sur la validation

Spring MVC – Pratique – Validation

Autres annotations possibles en java :

- @Null / @NotNull
- @NotEmpty / @NotBlank
- @Size(min=?, max=?) / @Length(min=?, max=?)
- @Min / @Max
- @Future / @Past
- @Range
- @Pattern(regex=?)
- @Email
- @AssertTrue / @AssertFalse
- @Valid : exécute la validation récursivement sur l’objet associé

Spring MVC – Pratique

Étape 5 : Ajouter la validation sur les champs

- Ajouter le starter *spring-boot-starter-validation*
- Ajouter les validations sur les entités
  - **Person** : Nom et prénom non vides et max 50 chars
  - **Species** : nom commun et latin requis avec longueur max selon la taille de la colonne en base (50 et 120)
  - **Animal** : Doit avoir un nom non vide et une Species non nul
- Ajouter le @Valid sur la création et l'update (@PostMapping)
- Ajouter l'affichage d'erreur sur les template Thymeleaf

Références et Ressources

- Spring : Quickstart Spring MVC

- Doc Thymeleaf
- Doc Thymeleaf Spring